

Asymptotic

A Differential Audit

New evidence beyond the nine existing code reviews

Wolfram Language comparison, a false-domain certificate, flat-sector precision loss, and resolution-aware numerical checks

Repository	VladimirReshetnikov/Asymptotic
Pinned revision	921387e5ba1239bfda96e63e64e89bf63d9c41e6
Package version	1.8.0
Native kernel used	Wolfram Language 15.0.0, Linux x86-64
Review date	9 September 2026
Prepared for	Vladimir Reshetnikov

Principal result. A cleanly constructed inverse object for the source domain $x > 10$ can subsequently receive a successful certificate with root 2 and verification interval $[1, 3]$ when unrelated ambient assumptions are active. A narrowly scoped patch rejects this case and preserves a valid control. Independently, flat-sector multiplication loses precision when one input is made more informative; a proved, tested boundary-projection repair restores the appropriate bound.

Evidence boundary. This report includes successful focused native experiments, source analysis, and independently executed Python checks. It does not claim a full-suite run, a complete formal verification, or measured whole-package performance. Proposed tests and optimizations are explicitly distinguished from executed checks.

Abstract

This audit reviews a pinned revision of the Asymptotic repository while treating its nine existing code reviews as an exclusion baseline. It contributes four specific deltas. First, the previously recognized ambient-assumption problem is sharpened to a native reproduction of an incorrect source-domain certificate for an already constructed object. Second, the flat-sector product algorithm prematurely erases exponential degrees: increasing one operand’s sector depth can worsen the algebraic factor in the reported tail. A boundary-sensitive replacement is proved and tested. Third, a native numerical inverse check is shown to return an inexact zero ratio with roughly minus 800 digits of accuracy; this is not an exact-zero error, but it needs an explicit resolution status. A normalized correction equation resolves the example without subtracting nearly equal source roots. Fourth, native Wolfram 15 already supplies the first three Dirichlet terms of the zeta function, contrary to an overbroad source comment. The package’s genuine additional value in that example is its explicit coordinate and quantitative tail bounds. The report develops a concrete fixed-order derivative bound for those tails, compares the broader architecture with native capabilities, and supplies narrow patches, reproductions, test specifications, and independent mathematical checks.

Contents

1	Executive assessment	3
2	Scope, reproducibility, and exclusion of old findings	4
2.1	What was examined and executed	4
2.2	How duplication was controlled	4
2.3	Evidence vocabulary	5
3	Architecture and comparison with built-in Wolfram Language	5
3.1	Compare mathematical contracts, not only finite expressions	5
3.2	Capability map	6
3.3	Native controls that prevent unfair comparisons	7
3.4	Sound design choices worth preserving	8
4	N1: a false source-domain certificate from ambient assumptions	8
4.1	The witness	8
4.2	Mechanism and logical error	9
4.3	Narrow mitigation, observed controls, and its limits	10
4.4	Tests that specifically exercise the proof boundary	10
5	N2: exponential-degree erasure weakens flat products	10
5.1	Representation and the current projection	10
5.2	A reproduced anti-refinement family	11
5.3	A boundary-sensitive projection theorem	12
5.4	Patch behavior and independent validation	13

6	N3: a zero numerical ratio with no useful resolution	13
6.1	Observed output and its correct interpretation	13
6.2	The true error is strictly positive	14
6.3	A resolution-aware public diagnostic	14
6.4	Normalize the correction before solving	15
7	N4: a version-stamped correction to the zeta comparison	15
8	Performance opportunities derived from N2 and N3	16
8.1	Avoid computing coefficients that only need an envelope	16
8.2	Separate proof cost, coefficient cost, and diagnostic cost	16
9	A concrete extension: differentiable Dirichlet tail contracts	17
9.1	Do not confuse source derivatives with local-coordinate derivatives	18
10	Remediation plan and integration acceptance criteria	18
10.1	First: protect certificate semantics	18
10.2	Second: repair the flat tail before optimizing its computation	18
10.3	Third: classify diagnostic resolution	19
10.4	Fourth: correct and strengthen native comparisons	19
10.5	Then: add the concrete derivative-tail contract	19
11	Artifacts and how to reproduce the work	19
12	Conclusion	20
A	Compact evidence ledger	20

1 Executive assessment

The repository is valuable as a real-asymptotic computation layer whose results carry more structure than a printed approximation: selected branches, local coordinates, retained coefficient blocks, remainder information, source equations, and, on some paths, refinement state. Its scope is substantially broader than its original inverse-power-series use case. The inspected sources include ordinary power-log calculus, exact inverse cores, reciprocal-logarithmic coordinates, Fourier-polynomial coefficients, finite flat sectors, native special-function adapters, and exact rational interval certificates. [1, 9, 10, 13, 14, 16]

It should not be evaluated against an artificially weak description of Wolfram Language. Native **Series** already handles some fractional powers and logarithms; **Asymptotic** uses scales beyond ordinary Taylor series; **InverseSeries** performs reversion; and **AsymptoticSolve** supplies local implicit solutions, including real-branch and multivariable problems. The appropriate comparison is the complete contract needed by a workflow, not the presence or absence of the word “asymptotic.”[21, 24, 23, 25]

ID	Priority	New or materially sharper result	Evidence
N1	High: certificate integrity	A stored domain is incorrectly marked verified on an interval disjoint from it. The object was constructed outside the hostile ambient context.	Native witness, clean control, patched rejection and valid control.
N2	Medium: precision and wasted work	Flat-tail multiplication collapses degree information too early. Depths $(1, 1)$, $(1, 2)$, $(1, 3)$ produce tail powers -4 , -6 , -8 instead of the available -1 .	Native baseline and patch; proof; 4,000 independent bound-model checks.
N3	Medium: diagnostic resolution	A numerical error ratio prints as zero but has accuracy about -799.89 . Direct subtraction cannot resolve the requested scale at the working precision.	Native observation; independent 60- and 950-digit comparisons; stable correction-coordinate calculation.
N4	Low: capability claims	Native Asymptotic and the finite part of Series already recover the three-term zeta example.	Version-stamped native comparison and package tail-bound comparison.

These findings have different meanings. N1 is an incorrect certificate claim. N2 returns a *valid but unnecessarily weak* asymptotic bound, not a false bound. N3 does not accuse Wolfram’s arbitrary-precision arithmetic of treating an inexact zero as an exact number: its accuracy metadata correctly reveals the loss of information. N4 is a documentation and comparison correction, not a reason to remove a useful dedicated adapter.

The highest-value development path from this audit is correspondingly narrow: isolate the certificate proof context; retain exponential degree until the last precision projection; make numerical resolution inspectable; and use native algorithms as independent coefficient oracles on genuinely overlapping families. The later derivative-tail construction is a concrete extension of one existing family, not a repetition of a general transseries wishlist.

2 Scope, reproducibility, and exclusion of old findings

2.1 What was examined and executed

The review is pinned to 921387e5ba1239bfda96e63e64e89bf63d9c41e6. The standalone package downloaded by the native evaluator contained 574,894 bytes and 38 inlined source modules. The native runtime identified itself as

```
15.0.0 for Linux x86 (64-bit) (May 6, 2026).
```

The source investigation covered the review/status records, README, standalone builder, series calculus and refinement paths, certificate and numerical-check paths, source-coordinate reductions, flat and Fourier layers, reciprocal-log operations, Gamma inverse operations, and several special-function layers. The package was loaded using `URLDownload` followed by local `Get`, not direct evaluation of a compressed HTTP response.[1, 8, 2]

The successful native observations were deliberately small. They include the certificate witness and controls, flat products before and after the proposed replacement, the numerical-resolution example, classical series/reversion controls, zeta comparisons, and two direct calls to `AsymptoticSolve`. The latter returned unevaluated on the tested irrational-power and flat-inverse requests. That observation is not a proof that no native reformulation can solve those problems.

Several other tool attempts returned HTTP 502 or network errors. Those attempts provide no evidence that a package function is slow, incorrect, or unsupported. No runtime or peak-memory number is inferred from a connector failure. The accompanying JSON records successful observations as a faithful transcription, not as a raw kernel transcript or a complete test report.

Independent local checks use SymPy for four flat-inverse coefficients and mpmath for numerical comparisons. A small, separately implemented asymptotic bound model tests 4,000 deterministic randomized projections and two exact-zero edge cases. These checks validate the stated mathematical transformations and model behavior; they do not execute the Wolfram package. An additional 28 high-precision numerical cases check the differentiated zeta-tail formula, and 17 synthetic-fixture checks exercise the patch utility.

2.2 How duplication was controlled

The maintained status ledger consolidates the existing reviews and identifies both unresolved findings and recently completed fixes. It was used as the primary exclusion map. Candidate topics were screened in the full article text of reviews 1–6, 8, and 9; the relevant detailed findings and engineering sections of review 7 were read. Zeta mentions were screened across all nine, including review 7’s mathematical-capabilities section. This was targeted novelty screening, not a claim to have re-proved every earlier report. [3, 2, 4, 5, 6, 7]

Delta	Closest existing material	Additional contribution here
N1	C05; review 1 A04 and review 8 F01: ambient assumptions can escape retained provenance.	A clean object, changed ambient state only during verification, and an explicitly false whole-interval domain assertion. A tested mitigation at the proof step, with its limited scope stated.

Delta	Closest existing material	Additional contribution here
N2	Review 7’s general information-monotonicity recommendation; P06’s resource budget discussion.	A precise flat-tail mechanism, an infinite family of weakening bounds, actual outputs, a proof of the repair, and an implementation replacement.
N3	D05’s checked-evaluation discussion; the existing separation between numerical evidence and certification.	A concrete loss of about 800 digits in the <i>inverse-check</i> ratio, correct interpretation of the inexact zero, and a stable normalized correction equation.
N4	Review 1 lists zeta/Lerch defining-sum constructions.	An executed native comparison showing the same finite terms, while distinguishing the package’s stronger tail metadata.
Extension	X04’s broad differentiation/re-expansion opportunity.	An explicit all-fixed-order derivative-tail formula for the Dirichlet family, with conditions, proof, and a standalone Wolfram function.

This report does *not* re-report the dense native-export allocation problem, the nested fractional-power guard, logarithmic-degree loss in native export, generic fixed-margin refinement, a general resource-budget redesign, the head rename/migration issue, CI proposals, license metadata, the primitive flat-rate GCD proposal, or broad complex/multivariate/uniform-asymptotic extensions. In particular, the ledger records completed fixes for the dense-export and fractional-power issues; these are not presented as current discoveries. [2]

2.3 Evidence vocabulary

“Observed” denotes a successful native or independent execution recorded in the bundle. “Source-derived” denotes a consequence of inspected control flow. “Proved” denotes a mathematical argument written below. “Proposed” denotes a change whose full integration remains work for the repository. A proposed regression file is not silently counted as an executed test suite.

3 Architecture and comparison with built-in Wolfram Language

3.1 Compare mathematical contracts, not only finite expressions

An asymptotic result can be regarded as a tuple

(germ, scale, finite part, error class, conditions, evidence).

Two outputs can have the same finite expression and substantially different usefulness. One may retain the original equation, another may be easier to compose with native `SeriesData`, and a third may carry an explicit pointwise tail bound. Conversely, a more elaborate result object is not better when its additional asserted evidence is wrong. N1 illustrates the latter case; N4 illustrates the former.

The ordinary series layer records a representation of the form

$$b(y) + a(y) \left(\sum_{\beta < H} w(y)^\beta P_\beta(\log w(y)) + \mathcal{O}\left(w(y)^\rho M(w(y))^d\right) \right), \quad M(u) = 1 + |\log u|,$$

with a positive local coordinate and an exact prefactor. The source explicitly separates relative jet precision from that prefactor. Other families deliberately use other structures: a retained Lambert core, finite Fourier modes, or separate exponential-sector and inner-power truncations. They should not be flattened into one native rational-lattice object merely because their finite parts can be printed as a sum.[9, 11, 12, 15]

Native **SeriesData** represents powers on a rational lattice through an integer denominator. That statement is about the data object’s exponent encoding, not a claim that **Series** can return only a single simple **SeriesData** or cannot produce factored expressions. The observed native zeta output is a useful counterexample to the latter misconception. [22]

3.2 Capability map

Task	Native capabilities	Repository contribution / boundary
Ordinary local series	Series , SeriesCoefficient , Normal , composition and reversion of suitable series.	Explicit power-log models and retained source/remainder metadata; ordinary controls should agree.
Fractional powers and logs	Series handles some fractional and logarithmic expansions; Asymptotic infers richer scales.	An explicit ordered real-exponent algebra and complete logarithmic blocks, rather than requiring one rational lattice for all exponents.
Implicit/inverse expansions	InverseSeries ; local solutions from AsymptoticSolve , with real/complex and multivariable forms.	Specialized real inverse methods, branch/source information, power observables, and refinement of supported nonclassical scales.
Exact inverse cores	Native ProductLog and special functions are useful building blocks.	Retains an exact core while ordering corrections relative to it; avoids replacing the core by a numerically inadequate finite log expansion.
Gamma/Barnes inverse families	Native forward asymptotics and Lambert functions can support hand-derived inversion.	Family-specific inversion and coefficient blocks; supported calculus is intentionally narrower than ordinary jets.
Oscillatory log coefficients	Native trigonometric/exponential algebra and asymptotic tools can aid derivations.	A finite Fourier-polynomial coefficient algebra with exact frequencies and explicit frequency budgets.
Flat sectors	Native expressions can represent exponentials; individual asymptotic and transformed problems may be solvable.	A separate sector degree, exact zero sector, inner bounds, and flat-specific operations; N2 concerns precision inside this existing machinery.
Special functions	Broad native symbolic function coverage and asymptotics.	A real-domain adapter layer and selected explicit identities/tail models; function coverage alone does not establish a unique advantage.

Task	Native capabilities	Repository contribution / boundary
Zeta defining sum	Native 15.0.0 supplies the tested three finite terms through both <code>Asymptotic</code> and <code>Normal[Series[...]]</code> .	Explicit $w = e^{-x}$ scale, positive lower tail, and a quantitative upper tail bound.
Numerical comparison	Arbitrary precision, <code>FindRoot</code> , and precision/accuracy tracking.	Inverse-aware checks and retained original equations; N3 needs an additional resolution classification, not a replacement for native arithmetic.
Interval evidence	Native exact arithmetic and symbolic processing are usable primitives.	A custom outward-rational enclosure implementation and a local root certificate; its domain proof must be isolated from ambient state.
General analysis	Native asymptotic integral, transform, equation and related interfaces cover a broad surrounding system.	A specialized library, not a replacement for that surrounding system. No new blanket adapter wishlist is proposed here.

Sources for this map are the public guide and inspected modules, together with official documentation for the native functions. [1, 9, 10, 11, 12, 13, 14, 15, 18, 19, 20, 17, 16, 21, 23, 25, 24]

3.3 Native controls that prevent unfair comparisons

Two inexpensive native controls were executed:

```
Series[x^x, {x, 0, 3}]
InverseSeries[Series[x + x^2, {x, 0, 5}], y]
```

Their returned coefficient data are respectively

$$\{1, \log x, (\log x)^2/2, (\log x)^3/6\}, \quad O(x^4) \text{ endpoint,}$$

$$\{1, -1, 2, -5, 14\}, \quad y - y^2 + 2y^3 - 5y^4 + 14y^5 + O(y^6).$$

The first is a native `SeriesData` whose displayed coefficients themselves contain logarithms. It directly rules out the claim that logarithmic coefficients alone make the package unique. It is not used here to infer a stronger mathematical logarithmic-tail contract than the native object states.

A more specialized test was

```
AsymptoticSolve[x + x^Sqrt[2] == y,
  {x, 0}, {y, 0, 3}, Reals]
```

which remained unevaluated on the tested kernel. The package's explicit inverse constructor with the exclusive cutoff 3 returned

$$g(y) = y - y^{\sqrt{2}} + \sqrt{2} y^{2\sqrt{2}-1} + \left(-3 + \frac{1}{\sqrt{2}}\right) y^{3\sqrt{2}-2} \\ + \left(-4 + \frac{17\sqrt{2}}{3}\right) y^{4\sqrt{2}-3} + \text{its recorded remainder.}$$

Every displayed exponent is below 3. The direct native request for $x + e^{-1/x} = y$ at the positive zero endpoint likewise remained unevaluated, whereas the dedicated flat constructor supplied sector expansions used below. These are useful concrete capability deltas, but not claims of impossibility for native transformations, manually introduced markers, or other algorithms. The numeral 3 is not a matched-order benchmark: the package call uses an exclusive exponent cutoff, whereas the native implicit solver’s approximation order is not necessarily a polynomial-degree cutoff. The comparison records what the stated direct requests do, not equivalence of their order conventions.

3.4 Sound design choices worth preserving

The source contains substantial safeguards that should survive optimization. A generic magnitude remainder is not automatically differentiable. The ordinary calculus checks derivative contracts. Exact source transformations are treated differently from approximate exponentiation. Gamma and Barnes inverse scales have explicit operation restrictions instead of pretending to be ordinary polynomial-log jets. The Fourier layer checks real conjugacy relations and uses frequency limits as hard resource bounds rather than silently deleting modes.[9, 10, 12, 13]

The exact special-function normalization layer deserves particular credit. For example, a half-integer modified Bessel function can contain both dominant and subdominant exponentials even when a dominant asymptotic series terminates. The implementation normalizes exact identities before finite asymptotic truncation, and it retains the second exponential where justified. Similar care appears in reflected error functions and finite hypergeometric identities. A review should not propose “recognize exact termination” without preserving that distinction.[19]

The standalone builder also has meaningful engineering safeguards: it parses relevant source-level imports, records module hashes, rejects repeated/cyclic or out-of-scope dependencies, and validates the assembled result before an atomic replacement. These features are present, not new recommendations. [8]

4 N1: a false source-domain certificate from ambient assumptions

4.1 The witness

Construct the inverse object under a neutral ambient context:

```
s = Block[{$Assumptions = True},
  AsymptoticInverse[AsymptoticInverse[
    ConditionalExpression[x, x > 10],
    {x, Infinity}, {y, 3}]];
```

The object retains `SourceDomain -> x > 10`. Now perform only the certificate call inside a different context:

```
Block[{$Assumptions = x > 10},
  AsymptoticInverse[InverseCertificate[s, 2,
    "Interval" -> {1, 3}, "Center" -> 2,
    "MaxRefinements" -> 0]]
```

The unpatched native result included

```
"Certified" -> True
```

```
"RootEnclosure" -> {2, 2}
"VerificationInterval" -> {1, 3}
"CertifiedSourceDomain" -> x > 10
"SourceDomainVerified" -> True
"CertifiedErrorBound" -> 0
```

The same call with `$Assumptions = True` returned `Failure["OutsideBranch", ...]`. Thus the defect is not explained by an already corrupted object or by an ambiguous source branch chosen at construction.

The distinction between the equation and its domain is crucial. The number 2 really is a root of the unrestricted equation $x = 2$, so its residual and zero error bound for that unrestricted equation are unsurprising. But there is no root of the restricted problem $x = 2, x > 10$. Moreover, *every* point of the verification interval violates the retained domain. The incorrect claim is the certificate's branch/domain assertion, not the elementary arithmetic.

4.2 Mechanism and logical error

The relevant function is `certPositiveCondition` in `InverseCertificates.wl`. Before interval comparisons, it performs

```
c = TimeConstrained[
  Quiet[Refine[condition, Element[x, Reals]]],
  1, condition];
```

and accepts `c === True`. Passing a second assumption argument to `Refine` does not, by itself, neutralize ambient assumptions. The following native probe returned `True`; analogous probes of `Simplify` and `FullSimplify` did too:

```
Block[{$Assumptions = x > 10},
  Refine[x > 10, Element[x, Reals]]]
```

This is normal assumption-aware simplification behavior. The mistake is using its conclusion as a statement about all points of an independently specified interval.^[16, 26, 4, 7]

The proof obligation is

$$\forall t \in [1, 3], \quad t > 10.$$

The simplifier is instead allowed to use a hypothesis equivalent to the proposition being checked. The valid implication

$$x > 10 \implies x > 10$$

does not establish the quantified interval claim. The error occurs before the otherwise useful exact-rational interval arithmetic becomes relevant.

This is why the new witness is materially stronger than merely showing that a constructor records `Assumptions -> True` after using a parameter hypothesis. The proof stage itself can turn a false branch assertion into a successful evidence record for an unchanged, correctly constructed object. Adding the hostile assumption to the stored domain would not fix this quantifier error: the domain would still be false on the interval.

4.3 Narrow mitigation, observed controls, and its limits

The supplied patch changes the simplification step to

```
c = Block[{$Assumptions = True},
  TimeConstrained[
    Quiet[Refine[condition, Element[x, Reals]]],
    1, condition];
```

The exact old anchor occurred once in the downloaded standalone file. A semantically identical replacement was applied in the native test. The hostile `[1, 3]` call then returned `Failure["OutsideBranch", ...]` with `Certified -> False`. A positive control with target 12, center 12, and interval `[11, 13]` still returned a certificate with enclosure `{12, 12}`.

The patch is intentionally described as a *mitigation of this proof-step leak*, not a repository-wide cure for ambient assumptions. The broad construction/operation-context problem is already recorded in C05. The supplied change neither captures all public-entry hypotheses nor validates every other symbolic oracle in the package.

For the certificate subsystem, the stronger integration invariant should be:

After an inverse object and exact numerical target are fixed, changing ambient assumptions must not create additional unconditional certificate claims. Only explicitly admitted and correctly scoped evidence may discharge a proof obligation.

In particular, a predicate being verified on an interval must not be available as an unexamined pointwise hypothesis for its own proof. Parameter conditions must either be checked after specialization or remain explicit conditions of a conditional result; they cannot be silently treated as numerical interval facts.

4.4 Tests that specifically exercise the proof boundary

The bundle supplies the false-domain witness, a clean-context control, and a valid interval control. Further integration tests should vary the domain's Boolean structure: strict and nonstrict inequalities, conjunctions, disjunctions, negations, rational endpoints, and predicates whose truth differs between interval endpoints. These are not a second generic request for assumption tests: their acceptance criterion is the truth of a whole-interval certificate after construction has already completed.

A particularly useful negative control uses ambient assumptions that conflict with the interval rather than with the stored source expression. It catches precisely the difference between symbolic implication and universal interval containment. Tests should inspect the structured certificate fields, not only whether an expression simplifies or an exception is raised.

5 N2: exponential-degree erasure weakens flat products

5.1 Representation and the current projection

The flat layer uses an exponential marker

$$E(u) = \exp(-c/u^p), \quad c, p > 0, \quad u \rightarrow 0^+,$$

and separate finite power-log coefficients. An object with sector depth N retains sectors $0, \dots, N$ and a complete tail of the form

$$E(u)^{N+1} \mathcal{O}(u^\rho M(u)^d).$$

Unknown inner coefficient errors remain in their original sectors. This is an important distinction already implemented by the package.[14, 15]

For input depths N_a, N_b , `flatOpsMultiplyData` chooses $N = \min(N_a, N_b)$, computes the finite sector convolution, and collects all omitted contributions into a tail beginning at E^{N+1} . Its current loop replaces every extra exponential factor by the inequality $E^q \leq 1$ and then combines the remaining algebraic bounds. The same loss happens for a tail times a nonzero sector and for a product of two tails.[15]

This is conservative but can be extremely wasteful. A factor E^q with $q > 0$ is not merely bounded by 1: it beats *every fixed* inverse power and logarithmic factor. Erasing q before comparing algebraic exponents loses exactly the information needed to avoid polluting the first omitted sector.

5.2 A reproduced anti-refinement family

Let g be the positive local inverse of

$$f(x) = x + e^{-1/x}, \quad x \rightarrow 0^+,$$

and put $E = e^{-1/y}$. Formal inversion, or direct substitution, gives

$$g(y) = y - E + y^{-2}E^2 + \frac{y - 3/2}{y^4}E^3 + \frac{y^2 - 4y + 8/3}{y^6}E^4 + \dots. \quad (1)$$

The four coefficients were independently generated by

$$C_k(y) = \frac{(-1)^k}{k!} e^{k/y} \frac{d^{k-1}}{dy^{k-1}} e^{-k/y}.$$

For the argument below, only the first two coefficients are necessary.

Construct s_1 with depth 1 and s_N with depth N , then multiply them:

```
a = AsymptoticInverse'AsymptoticFlatInverse[
  x + Exp[-1/x], {x, 0}, {y, 1}];
b = AsymptoticInverse'AsymptoticFlatInverse[
  x + Exp[-1/x], {x, 0}, {y, N}];
p = AsymptoticInverse'FlatSeriesMultiply[a, b];
```

The native outputs for $N = 1, 2, 3$ all have the same finite expression,

$$y^2 - 2yE,$$

but their tail pairs are

$$\{-4, 0\}, \quad \{-6, 0\}, \quad \{-8, 0\},$$

respectively, each multiplied by E^2 . Increasing the information in the second operand makes the reported common-depth product *less* precise.

For this model the constructor's tail at depth N is

$$E^{N+1} \mathcal{O}(y^{-2N}).$$

The product of the two input tails therefore contributes

$$E^{N+3}\mathcal{O}(y^{-2N-2}).$$

The current projection weakens this to $E^2\mathcal{O}(y^{-2N-2})$, which dominates the other projected algebraic envelopes and explains the observed progression. Higher finite coefficients have lowest powers no worse than $-2N + 2$; the $k - 1$ differentiations in C_k lower the power by at most $2(k - 1)$. Thus the same mechanism predicts the family $\rho_{\text{old}} = -2N - 2$ for all $N \geq 1$.

The available bound is much better:

$$g(y)^2 = y^2 - 2yE + E^2\mathcal{O}(y^{-1}). \quad (2)$$

Indeed, squaring (1) shows that the coefficient of E^2 is $1 + 2/y$. Equally, the depth-1 uncertainty $E^2\mathcal{O}(y^{-2})$ multiplied by the exact zero sector y produces $E^2\mathcal{O}(y^{-1})$. Terms with extra exponentials do not require a worse algebraic power. The old and improved scale estimates at $y = 10^{-3}$, $N = 3$, differ by the factor 10^{21} . This is a comparison of asymptotic scales, not of certified numerical constants.

5.3 A boundary-sensitive projection theorem

Lemma 5.1. *For $\delta, c, p > 0$ and fixed real a, b ,*

$$e^{-\delta c/u^p} u^a M(u)^b \longrightarrow 0 \quad \text{as } u \rightarrow 0^+.$$

Proof. The logarithm of the positive expression is $-\delta c/u^p + a \log u + b \log M(u)$. The first term tends to minus infinity faster than the absolute values of the other two grow. Exponentiating gives the claim. $\square$

Write the two input objects abstractly as

$$A = \sum_{i=0}^{N_a} E^i A_i + E^{N_a+1} R_a, \quad B = \sum_{j=0}^{N_b} E^j B_j + E^{N_b+1} R_b.$$

Each coefficient may itself carry an inner power-log uncertainty. Bounds below apply to the whole coefficient, not just its displayed finite part. Set $N = \min(N_a, N_b)$ and $D_k = \sum_{i+j=k} A_i B_j$.

Proposition 5.2. *At common retained depth N , the algebraic envelope at the first omitted sector E^{N+1} need only combine the following contributions:*

$$D_{N+1}, \quad \mathbf{1}_{N_a=N} R_a B_0, \quad \mathbf{1}_{N_b=N} R_b A_0. \quad (*)$$

If these have a finite power-log bound $\mathcal{O}(u^\rho M^d)$, the complete omitted product tail is $E^{N+1}\mathcal{O}(u^\rho M^d)$. If every contribution in $()$ is exactly zero but a later omitted contribution may be nonzero, the conservative fallback $E^{N+1}\mathcal{O}(1)$ is valid. An exact-zero tail is justified only when all omitted contributions are exact zero.*

Proof. Expand the product. Finite convolution terms omitted from the retained result have degree at least $N + 1$. A term $R_a B_j$ has degree $N_a + 1 + j$; it has degree exactly $N + 1$ precisely when $N_a = N$ and $j = 0$. The analogous assertion holds for $R_b A_i$. The tail product has degree $N_a + N_b + 2 > N + 1$.

Every remaining contribution has at least one additional positive integer power of E . By Lemma 5.1, its fixed power-log envelope can be absorbed into any chosen finite power-log envelope at degree $N + 1$. There are finitely many coefficient contributions and finitely many tail-product bounds. Their constants can therefore be combined. If the boundary is empty, choose the envelope 1 rather than claiming that no later contribution exists. $\square$

The proposition does not move an unknown inner error to a later sector. Such an error is included in the bound of its original A_i or B_j and therefore in the appropriate D_k . This is essential: simply forgetting inner errors would be an unsound and quite different change.

5.4 Patch behavior and independent validation

The supplied replacement implements $(*)$ after the existing convolution. It preserves the rest of the function, including compatibility checks, coefficient arithmetic, domain combination, and derivative provenance. It also explicitly handles an empty boundary with a possible later tail.

On the same three native cases, the patched routine returned the unchanged finite expression and tail pairs

$$\{-1, 0\}, \quad \{-1, 0\}, \quad \{-1, 0\}.$$

The independent Python model separately enumerates all tail contributions with their exponential degrees and compares them with the optimized boundary rule. It passed 4,000 seeded random cases using rational power exponents, logarithmic degrees, unequal depths, and exact zeros, plus two explicit empty-boundary cases. This tests the projection algebra, not all of the upstream coefficient and domain machinery.

The change is intentionally modest. It *does not yet avoid* computing the full finite convolution and therefore does not claim a runtime speedup. Its purpose is to repair precision loss with a small reviewable change. The next section separates the additional work-elimination opportunity.

6 N3: a zero numerical ratio with no useful resolution

6.1 Observed output and its correct interpretation

For the depth-1 inverse above, the native call

```
AsymptoticInverse(InverseNumericalCheck[
  a, 1/1000, WorkingPrecision -> 50]
```

returned the following important fields:

```
"Error" -> 0.'62.69897000433602
"Ratio" -> 0.'-799.8899938021677
"RootResidual" -> 0.'63.
"RemainderScale" -> 2.57653587296114965... * 10^-863
"Approximation" -> 0.001'50.
"ReferenceRoot" -> 0.001'50.
```

Double-backtick notation specifies accuracy. The ratio is *not* an exact zero and is not a well-resolved zero. Its large negative accuracy means the calculation has no useful information at the scale needed to compare that ratio with a number of order 1. Native precision tracking is exposing, rather than concealing, the cancellation.[27]

The source uses `WorkingPrecision + 10` for the approximation and reference root, then subtracts them. The remainder scale is evaluated afterward. It is not used to decide whether the root subtraction has enough resolution. The output's statement that it is numerical evidence rather than an interval certificate is correct, but it does not replace a resolution assessment for this particular diagnostic.[17]

6.2 The true error is strictly positive

Let $y > 0$ be small enough that $q = y - e^{-1/y} > 0$, and set $E = e^{-1/y}$. Since $q < y$,

$$f(q) - y = -E + e^{-1/q} < 0.$$

Also $f'(x) = 1 + e^{-1/x}/x^2 > 0$ for $x > 0$, and $f(y) > y$. The unique positive root of $f(x) = y$ is therefore strictly between q and y . The true error $g(y) - q$ is positive, independently of any floating-point computation. Equation (1) gives its leading scale:

$$g(y) - q = \frac{E^2}{y^2} \left(1 + \mathcal{O}(Ey^{-2})\right).$$

At $y = 1/1000$,

$$\log_{10}(E^2/y^2) = -862.5889638065036553 \dots$$

Subtracting two roots of magnitude 10^{-3} to obtain an error of magnitude about 10^{-863} loses roughly 860 significant digits. A 50-digit requested root computation with ten guard digits cannot resolve it.

The independent 60-digit direct subtraction returned zero. The 950-digit calculation returned

$$2.576535872961149652190150499507352912722130573760632202117945887577139 \times 10^{-863},$$

with error divided by E^2/y^2 equal to 1 at the displayed 70-digit precision. These are numerical checks, not interval enclosures. Their interpretation is supported by the separate strict-positivity argument.

6.3 A resolution-aware public diagnostic

An improved check should expose at least the accuracy of the subtraction and of the normalized ratio, and distinguish a successful root solve from a resolved asymptotic-error comparison. A result such as

```
"ReferenceSolveStatus" -> "Converged"
"ErrorResolutionStatus" -> "Insufficient"
"RatioAccuracy" -> -799.8899...
"ResolvedErrorRatio" -> Missing["InsufficientResolution"]
```

would accurately summarize the native information already present. This is a proposed diagnostic schema, not an existing API.

A simple adaptive fallback can estimate the extra working precision needed from the observed ratio accuracy or, before subtraction, from

$$\log_{10} \frac{|g_{\text{approx}}|}{\text{remainder scale}} + \text{desired ratio digits} + \text{guard margin}.$$

For this example, roughly 870 digits are needed for a ten-digit ratio before adding a guard margin. This is a workload-dependent planning estimate, not a universal precision theorem. Repeated computations must confirm stabilization, and a maximum precision budget must be respected. A formal $\mathcal{O}$ scale is not a pointwise certified upper bound, so the adaptation must not label the result “certified.”

The better solution on some families is to avoid subtracting the source roots at all. That opportunity is concrete here.

6.4 Normalize the correction before solving

Write

$$g(y) = y - E + \frac{E^2}{y^2}w, \quad u = y - E + \frac{E^2}{y^2}w.$$

The desired error ratio is exactly w . Define

$$t = -\frac{E(1 - Ew/y^2)}{uy}, \quad \text{exprel}(t) = \frac{e^t - 1}{t}, \quad \text{exprel}(0) = 1.$$

Then $f(u) = y$ is equivalent, where the displayed denominators are nonzero, to

$$w = \frac{y(1 - Ew/y^2)}{u} \text{exprel}(t). \quad (3)$$

To verify it, observe that $-1/u + 1/y = t$ and divide $-E + E^2w/y^2 + Ee^t = 0$ by E^2/y^2 . The factor $(e^t - 1)/t$ removes the cancellation analytically before numerical evaluation.

At $y = 1/1000$, w is of order 1 even though the source error is of order 10^{-863} . Using a stable `expm1` evaluation in the included Python implementation, the normalized equation gives ratio 1 to the displayed 55 digits at 60-digit working precision. The direct 950-digit calculation agrees. This is a change of numerical coordinates, not artificial padding of input precision.

The method suggests a useful diagnostic extension: when an inverse object retains a leading omitted sector or a controlled residual model, solve for a normalized correction variable rather than always for the original source root. The eligibility checks must establish a meaningful nonzero scale and an appropriate local branch. This report supplies the exact scalar example and its numerical implementation, not a claim of a universal automatic normalization algorithm.

7 N4: a version-stamped correction to the zeta comparison

The introductory comment in `DirichletSpecialFunctions.wl` describes expansions from defining sums as unavailable from native `Series`. That is too broad for the inspected runtime. The following native request returned $1 + 2^{-x} + 3^{-x}$:

```
Asymptotic[Zeta[x], x -> Infinity, SeriesTermGoal -> 3]
```

The native `Series[Zeta[x], {x, Infinity, 3}]` result had a less convenient nested form,

```
1 + 2^SeriesData[x, Infinity, {-1}, -1, 4, 1]
+ 3^SeriesData[x, Infinity, {-1}, -1, 4, 1]
```

but applying `Normal` again gave $1 + 2^{-x} + 3^{-x}$. The package's corresponding three-term request returned the same finite part. These observations concern this particular version and input; they do not claim a native result for every transformed zeta or Lerch problem. [20, 24, 21]

The package additionally returned the positive local coordinate $w = e^{-x}$, a lower remainder bound 4^{-x} , and an upper bound

$$4^{-x} \left(1 + \frac{4}{x-1} \right), \quad x > 1. \quad (4)$$

These are meaningful additional outputs. They follow directly from the positive defining tail and the integral test; they are not obtained merely by dropping a nested native remainder.

The appropriate correction is therefore not “delete the adapter because the kernel can do it.” Instead, change the capability wording, add a native coefficient-equivalence test for this overlapping case, and document the dedicated adapter’s explicit error and coordinate contract. A benchmark should separately compare finite-term generation and generation of the richer bound record. Reporting one as a speed comparison with the other would compare different tasks.

There is also an oracle-quality benefit. A dedicated defining-sum implementation and the native `Asymptotic` route are independently useful for checking finite coefficients. But agreement of their finite parts does not prove the package’s remainder bound or source-domain claim. Those require the integral argument and the branch checks respectively.

8 Performance opportunities derived from N2 and N3

8.1 Avoid computing coefficients that only need an envelope

The current flat product computes all finite sector-pair products even though its output retains only through $N = \min(N_a, N_b)$. Proposition 5.2 shows that an improved implementation needs exact convolution coefficients only through sector $N + 1$: retained sectors and the first omitted boundary. Higher sectors need only the fact that their coefficients have the supported fixed power–log envelopes.[15]

For the reproduced depth pair $(1, N)$, the current rectangular convolution has $2(N + 1)$ candidate pairs. For $N \geq 2$, only five pairs have total degree at most 2. Three contribute to the retained sectors 0 and 1, and two contribute to the boundary sector 2. This constant-size coefficient workload is independent of how far the second operand was refined. The supplied precision patch still uses the original full convolution; the five-pair algorithm is the proposed second-stage optimization.

The difference matters when coefficients are large symbolic power–log polynomials. Multiplying and expanding a deep coefficient merely to learn a bound that an extra exponential will absorb is unnecessary work. It can also cause a local resource refusal unrelated to the information the requested output actually needs. This is a concrete demand-pruning opportunity, rather than a repetition of the existing proposal to rename or split `MaxTerms`.

A safe implementation should retain the following facts without materializing all later products: whether a coefficient is exact zero; its supported envelope class; whether a tail may be nonzero; and the exponential degree at which a contribution starts. It should not infer exact zero merely because the first omitted coefficient happens to cancel. The empty-boundary branch in the patch is specifically designed to avoid that error.

8.2 Separate proof cost, coefficient cost, and diagnostic cost

For this delta, three measurements would be informative: exact pair products actually computed, coefficient leaf counts before and after expansion, and the achieved tail pair. Wall time alone cannot detect a precision regression. The baseline returns a weaker tail when the second input is deeper, so a comparison that ignores the tail can falsely describe the computation as successful refinement.

For numerical diagnostics, measure the precision needed for a resolved *ratio*, not just for a converged source root. The normalized equation provides an example where the source-root coordinate is catastrophically inefficient for the diagnostic but a correction coordinate is well scaled. The independent 60/950-digit experiments establish the distinction in required precision; this audit does not claim a measured speedup factor.

The most useful acceptance comparison is therefore multi-dimensional: identical branch and finite part, an equal or stronger remainder, a resolved diagnostic when claimed, and reduced work. Existing broad performance and benchmarking recommendations need not be repeated to implement these specific comparisons.

9 A concrete extension: differentiable Dirichlet tail contracts

The existing roadmap already mentions differentiation and re-expansion. Here is a specific addition with a closed-form bound, rather than another request for a general differentiation engine. It applies to the existing zeta defining-sum family.[20, 2]

For integer $m \geq 2$ and $s > 1$, let

$$R_m(s) = \sum_{n=m}^{\infty} n^{-s}.$$

On every compact subset of $s > 1$, fixed-order differentiated series converge uniformly, since $(\log n)^j$ grows more slowly than every positive power of n . Termwise differentiation gives

$$(-1)^j R_m^{(j)}(s) = \sum_{n=m}^{\infty} (\log n)^j n^{-s} =: T_{m,j}(s) > 0.$$

The nonnegativity is useful evidence in its own right.

Proposition 9.1. *For an integer $j \geq 0$, $m \geq 2$, $s > 1$, and $s \log m \geq j$,*

$$T_{m,j}(s) \leq m^{-s} (\log m)^j \tag{5}$$

$$+ m^{1-s} \sum_{k=0}^j \frac{j!}{(j-k)!} \frac{(\log m)^{j-k}}{(s-1)^{k+1}}. \tag{6}$$

The right side is an explicit upper bound, not merely an asymptotic scale.

Proof. For $h(t) = t^{-s}(\log t)^j$, the logarithmic derivative is

$$\frac{h'(t)}{h(t)} = \frac{1}{t} \left(\frac{j}{\log t} - s \right).$$

Thus h is nonincreasing for $t \geq m$ under the stated condition. The integral test bounds its sum by $h(m) + \int_m^{\infty} h(t) dt$. With $v = \log t$,

$$\int_m^{\infty} t^{-s} (\log t)^j dt = \int_{\log m}^{\infty} e^{-(s-1)v} v^j dv.$$

Repeated integration by parts gives the finite sum in (6). Equivalently, the integral is $\Gamma(j+1, (s-1) \log m) / (s-1)^{j+1}$, with the upper incomplete gamma function. The finite expression avoids introducing another special-function bound evaluator. $\square$

For $j = 0$, the formula is exactly

$$R_m(s) \leq m^{-s} \left(1 + \frac{m}{s-1} \right),$$

which specializes to the observed package bound (4) at $m = 4$. For an affine source $s = ax + b$, the absolute j th derivative with respect to x acquires the factor $|a|^j$. The real source must still lie in the domain where the sum and monotonicity bound apply.

9.1 Do not confuse source derivatives with local-coordinate derivatives

The package’s ordinary derivative calculus is expressed in its recorded local coordinate. If $w = e^{-s}$, then

$$\frac{d^j}{dw^j} w^{\log n} = (\log n) \underline{j} w^{\log n - j},$$

where the underline denotes a falling factorial. It is not correct to attach a local-coordinate derivative contract merely because an x -derivative bound was written down.

For every fixed j , the absolute falling factorial is bounded by a polynomial of degree j in $\log n$ with fixed nonnegative coefficients. Applying Proposition 9.1 term by term to that polynomial gives, eventually as $w \rightarrow 0^+$,

$$\frac{d^j}{dw^j} R_m(-\log w) = \mathcal{O}(w^{\log m - j}).$$

This is the form compatible with the ordinary power-coordinate derivative contract. It supports every *fixed* derivative order; no uniform assertion for $j \rightarrow \infty$ is being made. For an affine $s(x)$, multiplication by the coordinate Jacobian supplies the expected source-variable scale.

The supplied `zeta_tail_derivatives.w1` implements the explicit source-exponent bound with its conditions. It treats $j = 0$ separately and, for positive j , starts at $\max(m, 2)$ so the $n = 1$ derivative vanishes without creating a 0^0 corner case. It is an independent proposal, not a patched public package API or an interval certificate. Integrating the contract should be restricted to the proved Dirichlet family rather than indiscriminately raising derivative orders on all special-function adapters.

10 Remediation plan and integration acceptance criteria

10.1 First: protect certificate semantics

Apply and review the narrow N1 change, regenerate the standalone artifact, and run the focused false-domain and valid-domain tests on both modular and standalone loading paths. The broader C05 work remains tracked in its existing location. The new acceptance target is that a neutral object cannot gain a false unconditional domain certificate merely by changing ambient state during verification.

A full release gate remains the repository’s existing maintenance topic; this report does not claim its focused controls replace one. The narrow patch should not be marketed as fixing every hypothesis-capture issue.

10.2 Second: repair the flat tail before optimizing its computation

Use the boundary projection to preserve the strongest directly available first-omitted-sector envelope. Validate unequal depths, nontrivial zero sectors, inner remainders, cancellations at the first omitted coefficient, exact scalar zero, logarithmic factors, and negative powers in the supported common chart. The proof covers fixed supported envelopes and commensurate sector degrees; it does not add new phase families.

Only after this precision behavior is stable should exact coefficient work above the first omitted boundary be pruned. The precise acceptance condition is unchanged retained coefficients and equal or stronger justified tails, with fewer unnecessary products. Request-budget behavior must remain explicit when a needed coefficient is genuinely expensive.

10.3 Third: classify diagnostic resolution

Preserve the existing numerical/certified distinction and additionally label whether the reported ratio is numerically resolved. A converged reference root and an unresolved error comparison should coexist in the result rather than one being mistaken for the other. Supply an adaptive-precision route with a hard limit and a correction-coordinate route only where a justified normalization is available.

The normalized flat example is a suitable regression because both the true error sign and its first scale are analytically known. A test must not merely assert that the printed ratio is zero or that `FindRoot` returned a number. It should examine numerical accuracy and agreement with the independent normalized coordinate.

10.4 Fourth: correct and strengthen native comparisons

Update the zeta comment and add the version-stamped overlapping-family check. Keep raw native `Series` output as well as its finite part: silently normalizing away all native remainder information would make a misleading representation comparison. For the package, retain its explicit upper/lower bounds and their conditions in the record.

The separate irrational-power and flat-inverse probes are useful positive examples of the package's practical scope. They should be recorded as direct native calls that remained unevaluated on a specific version, not as universal proofs that the native system cannot solve any equivalent formulation.

10.5 Then: add the concrete derivative-tail contract

Review Proposition 9.1, test the finite bound expression against high-precision differentiated defining tails in its admitted domain, and connect its local-coordinate form to the existing derivative-contract mechanism. Keep the argument and admissibility conditions alongside the implementation. This is an achievable extension of a present module, not a new general transseries platform.

11 Artifacts and how to reproduce the work

The archive contains the article source and compiled PDF, two exact-source patch replacements, a local-checkout patch utility, a native reproduction script, corrected-behavior test specifications, the derivative-bound proposal, an independent Python check program, and evidence/novelty records.

The patch utility defaults to a dry run. It checks the audited Git revision unless explicitly overridden, requires each source anchor exactly once, refuses to overwrite its backup, and writes each changed file atomically. It changes only a local checkout; it does not push commits or edit a remote repository. Run from the extracted bundle, for example:

```
python code/apply_patches.py /path/to/Asymptotic
python code/apply_patches.py /path/to/Asymptotic --write
# In that checkout, regenerate with the repository's documented
# validation/build_standalone.py workflow and check freshness.

wolframscript -file code/reproduce.wl \
  /path/to/Asymptotic/AsymptoticInverse.wl

python code/independent_checks.py
```

The native script records the loaded package path, hash, and kernel version. Run it once against the pinned baseline and once against the regenerated patched build. The complete `acceptance.wlt` file is a supplied integration specification; its individual core witnesses were exercised here, but the file as a whole was not run as a test suite during this audit.

The JSON evidence file is explicitly a transcription of successful connector outputs. The independent-results file is generated by the included Python program. Their separation prevents a mathematical oracle check from being mistaken for native execution of the repository. The novelty matrix identifies the closest old finding for each new contribution and records what was excluded.

12 Conclusion

The repository’s strongest direction is not to compete with every built-in symbolic-analysis feature. It is to provide reliable reusable results in selected real asymptotic scales, with explicit conditions and honest evidence. That position is well supported by the inspected architecture and by the practical irrational-power and flat-inverse examples.

The new work needed here is specific. A certificate must not borrow the very predicate it is meant to prove from ambient state. A flat-tail product must not erase exponential degree before deciding the algebraic envelope. A numerical comparison must not leave users to interpret an apparently zero ratio whose accuracy says it is unresolved. And native capability claims should be checked against current native outputs, not inferred from the inconvenience of their representation.

The supplied patches and proofs address two of these boundaries directly. The normalized correction example and the differentiated Dirichlet-tail bound show how further development can produce concrete mathematical improvements without repeating the existing broad roadmap. Their common principle is to preserve the information that gives an asymptotic result its meaning until the last operation that actually needs to discard it.

A Compact evidence ledger

Check	Result	Status
Ambient oracle	<code>Refine</code> , <code>Simplify</code> , <code>FullSimplify</code> each simplify $x > 10$ to True under ambient $x > 10$.	Native observed.
N1 baseline	Restricted inverse, target 2, interval $[1, 3]$: certificate succeeds only under the hostile ambient context.	Native observed.
N1 patch	Hostile invalid interval rejected; target 12 in $[11, 13]$ still certified.	Native observed.
N2 baseline	Depth pairs $(1, 1)$, $(1, 2)$, $(1, 3)$ yield E^2 tail powers $-4, -6, -8$.	Native observed.
N2 patch	Same finite parts, tail power -1 in all three cases.	Native observed.
Bound model	4,000 seeded random comparisons plus two exact-zero edge cases.	Independent, executed.
N3 baseline	Inexact zero ratio with accuracy -799.8899938021677 .	Native observed.

Check	Result	Status
N3 normalization	60-digit normalized ratio agrees with direct 950-digit subtraction at displayed precision.	Independent, executed.
Native controls	Logarithmic <code>Series</code> ; classical <code>InverseSeries</code> ; three zeta terms.	Native observed.
Specialized inverse	Two direct <code>AsymptoticSolve</code> calls remain unevaluated; package constructs the tested results.	Native observed.
Patch release	Full cross-family regression, all loaders, full native suite.	Not performed here.
Derivative bound	Closed finite upper bound and proof with explicit conditions; 28 independent numerical spot checks.	Proposed integration; independent checks executed.

References

- [1] Asymptotic repository, `README.md`, pinned revision 921387e5ba1239bfda96e63e64e89bf63d9c41e6.
- [2] Asymptotic repository, `docs/development/CODE_REVIEW_STATUS.md`, consolidated status and cross-review mapping.
- [3] Asymptotic repository, `code-review/README.md`, index of nine earlier reviews.
- [4] Review 1, `code-review-1/article/article-standalone.tex`, especially A04.
- [5] Review 7, `code-review-7/article/03-findings.tex`, correctness boundaries and information monotonicity.
- [6] Review 7, `code-review-7/article/04-engineering-and-roadmap.tex`.
- [7] Review 8, `code-review-8/article/asymptotic_repository_audit.tex`, especially F01.
- [8] Asymptotic repository, `validation/build_standalone.py`.
- [9] Asymptotic repository, `Kernel/SeriesOperations.wl`, especially `seriesData`, `seriesMake`, and `seriesDerivative`.
- [10] Asymptotic repository, `Kernel/SourceCoordinates.wl`.
- [11] Asymptotic repository, `Kernel/ReciprocalLogOperations.wl`.
- [12] Asymptotic repository, `Kernel/GammaInverseOperations.wl`.
- [13] Asymptotic repository, `Kernel/FourierCoefficients.wl`.
- [14] Asymptotic repository, `Kernel/FlatSectors.wl`, especially `flatConstruct` and its tail formula.
- [15] Asymptotic repository, `Kernel/FlatSectorOperations.wl`, especially `flatOpsMultiplyData` and `flatOpsJetBound`.
- [16] Asymptotic repository, `Kernel/InverseCertificates.wl`, especially `certPositiveCondition` and the closed-interval verification path.
- [17] Asymptotic repository, `Kernel/NumericalInverseChecks.wl`, especially `numericalInverseEvidence`.

- [18] Asymptotic repository, `Kernel/SpecialFunctionAdapters.wl`.
- [19] Asymptotic repository, `Kernel/SpecialFunctionIdentities.wl`.
- [20] Asymptotic repository, `Kernel/DirichletSpecialFunctions.wl`.
- [21] Wolfram Research, *Series*, official documentation, accessed 9 September 2026.
- [22] Wolfram Research, *SeriesData*, official documentation, accessed 9 September 2026.
- [23] Wolfram Research, *InverseSeries*, official documentation, accessed 9 September 2026.
- [24] Wolfram Research, *Asymptotic*, official documentation, accessed 9 September 2026.
- [25] Wolfram Research, *AsymptoticSolve*, official documentation, accessed 9 September 2026.
- [26] Wolfram Research, *Refine*, official documentation, accessed 9 September 2026; ambient behavior also tested directly.
- [27] Wolfram Research, *Accuracy*, official documentation, accessed 9 September 2026.